

CS121 Data Structures in C

07 Recursion

Outline

- Recursion
- Example1: Count down
- Example2: Factorial function
- Example3: Fibonacci numbers
- Example4: Tower of Hanoi

Recursion

- Recursion is a function that calls itself in its body.

Example: the factorial function

$$6! = 6 * 5 * 4 * 3 * 2 * 1$$

We could write:

$$6! = 6 * 5!$$

- Recursion is useful for problems that can be represented by a simpler version of the same problem.
 - ♦ To decompose a task into smaller subtasks. At least one of the subtasks is a smaller example of the same task.
 - ♦ The smallest example of the same task has a non-recursive solution.

Example: The factorial function

$$n! = n * (n-1)! \text{ and } 1! = 1$$

Example 1: the Idea of Recursion

- The countdown function

```
void count_down(int n)
{
    if (n <= 0)
        printf("\nLast One\n");
    else
    {
        printf("%d! ", n);
        count_down(n-1);
    }
}

int main ()
{
    count_down(3);
}
```

```
3! 2! 1!
Last One
```

Main Function

count_down(4)

n=4

```
void count_down(int n)
{
    if (n <= 0)
        printf("\nLast One\n");
    else{
        printf("%d! ", n);
        count_down(n-1);
    }
}
```

Output:

Main Function

count_down(4)

n=4

```
void count_down(int n)
{
    if (n <= 0)
        printf("\nLast One\n");
    else{
        printf("%d! ", n);
        count_down(n-1);
    }
}
```

n=3

```
void count_down(int n)
{
    if (n <= 0)
        printf("\nLast One\n");
    else{
        printf("%d! ", n);
        count_down(n-1);
    }
}
```

Output: 4

Main Function

count_down(4)

n=4

```
void count_down(int n)
{
    if (n <= 0)
        printf("\nLast One\n");
    else{
        printf("%d! ", n);
        count_down(n-1);
    }
}
```

n=3

```
void count_down(int n)
{
    if (n <= 0)
        printf("\nLast One\n");
    else{
        printf("%d! ", n);
        count_down(n-1);
    }
}
```

Output: 4

Main Function

count_down(4)

n=4

```
void count_down(int n)
{
    if (n <= 0)
        printf("\nLast One\n");
    else{
        printf("%d! ", n);
        count_down(n-1);
    }
}
```

n=3

```
void count_down(int n)
{
    if (n <= 0)
        printf("\nLast One\n");
    else{
        printf("%d! ", n);
        count_down(n-1);
    }
}
```

Output: 4 3

Main Function

count_down(4)

n=4

```
void count_down(int n)
{
    if (n <= 0)
        printf("\nLast One\n");
    else{
        printf("%d! ", n);
        count_down(n-1);
    }
}
```

n=3

```
void count_down(int n)
{
    if (n <= 0)
        printf("\nLast One\n");
    else{
        printf("%d! ", n);
        count_down(n-1);
    }
}
```

n=2

```
void count_down(int n)
{
    if (n <= 0)
        printf("\nLast One\n");
    else{
        printf("%d! ", n);
        count_down(n-1);
    }
}
```

Output: 4 3

Main Function

count_down(4)

n=4

```
void count_down(int n)
{
    if (n <= 0)
        printf("\nLast One\n");
    else{
        printf("%d! ", n);
        count_down(n-1);
    }
}
```

n=3

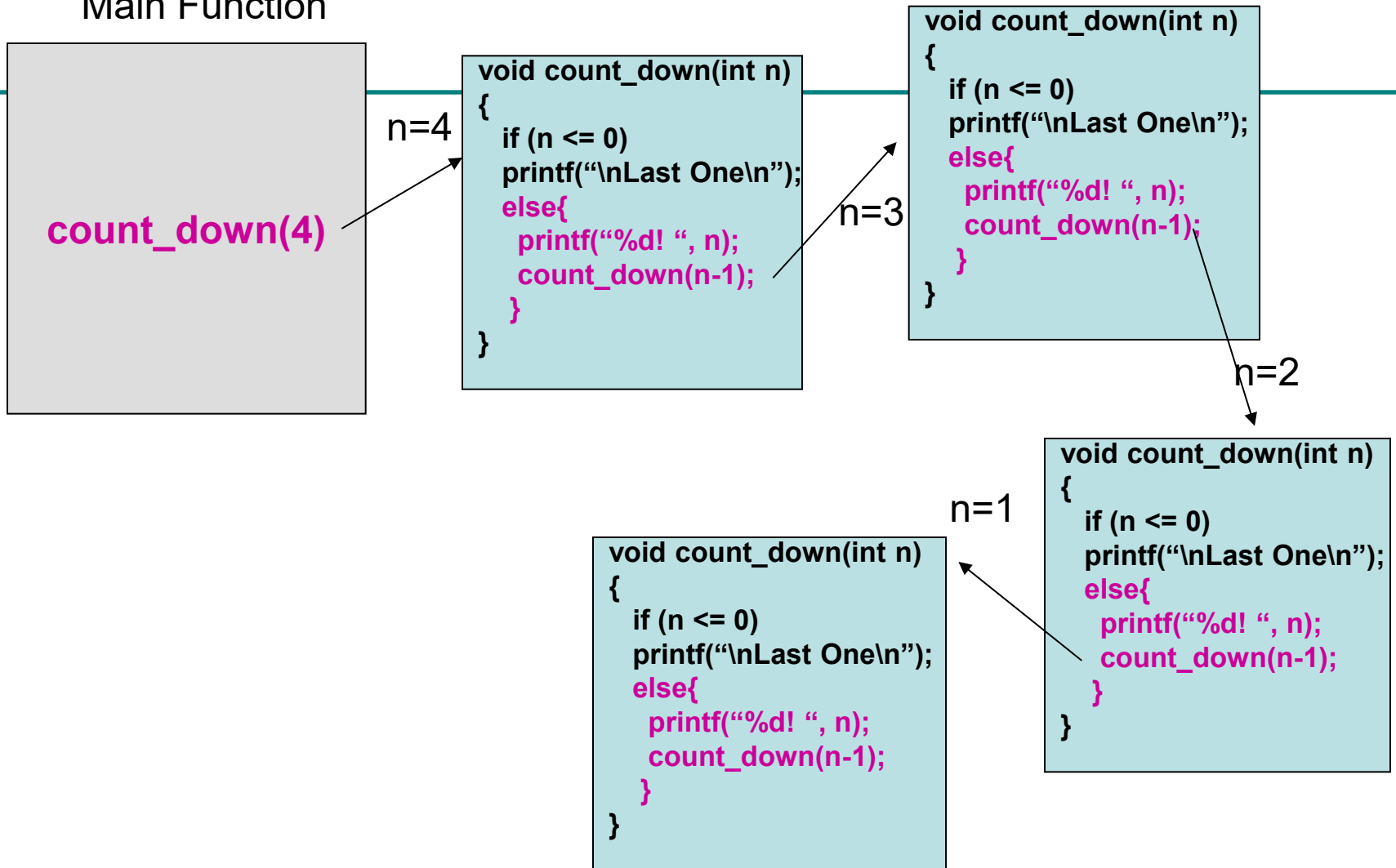
```
void count_down(int n)
{
    if (n <= 0)
        printf("\nLast One\n");
    else{
        printf("%d! ", n);
        count_down(n-1);
    }
}
```

n=2

```
void count_down(int n)
{
    if (n <= 0)
        printf("\nLast One\n");
    else{
        printf("%d! ", n);
        count_down(n-1);
    }
}
```

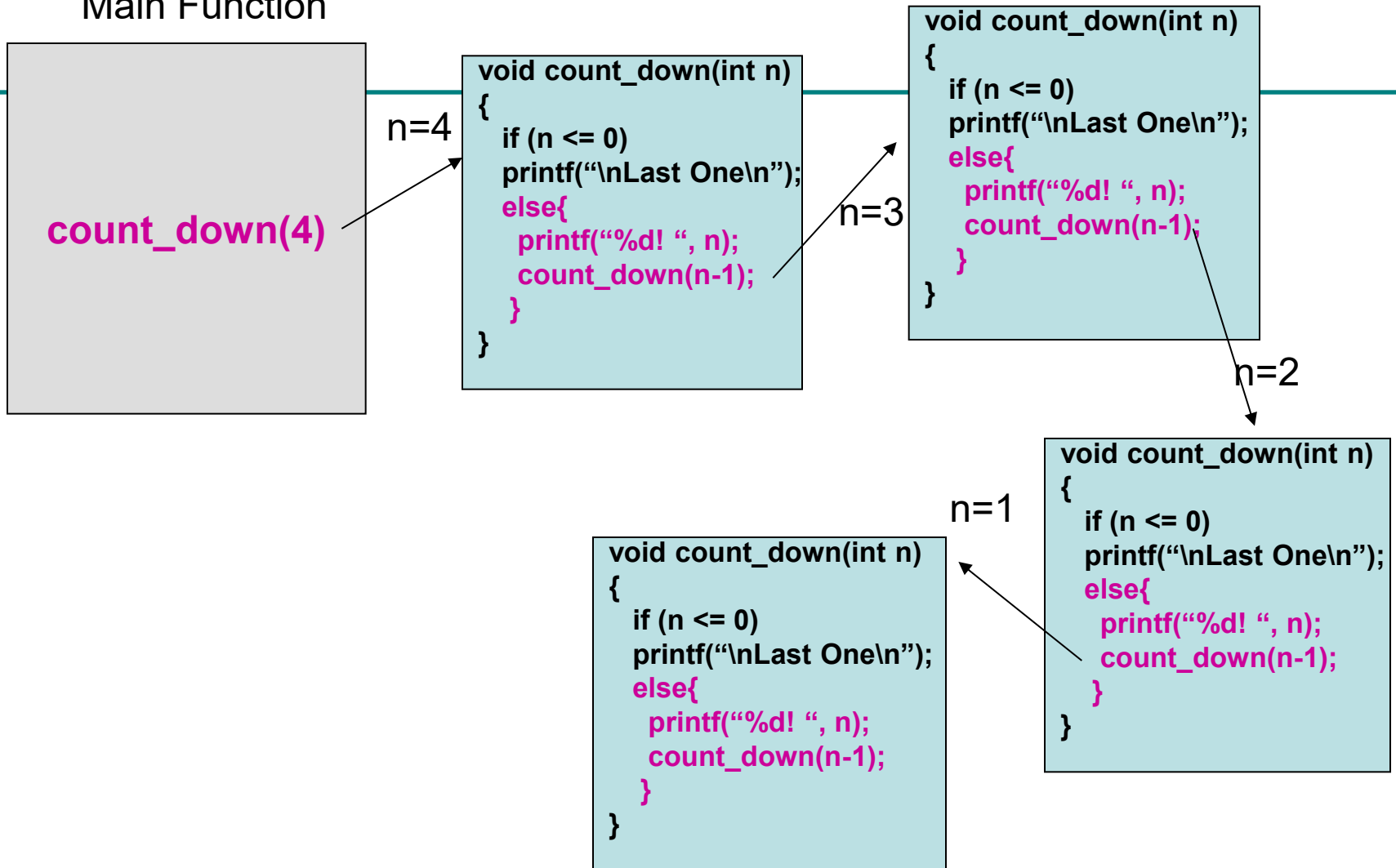
Output: 4 3 2

Main Function



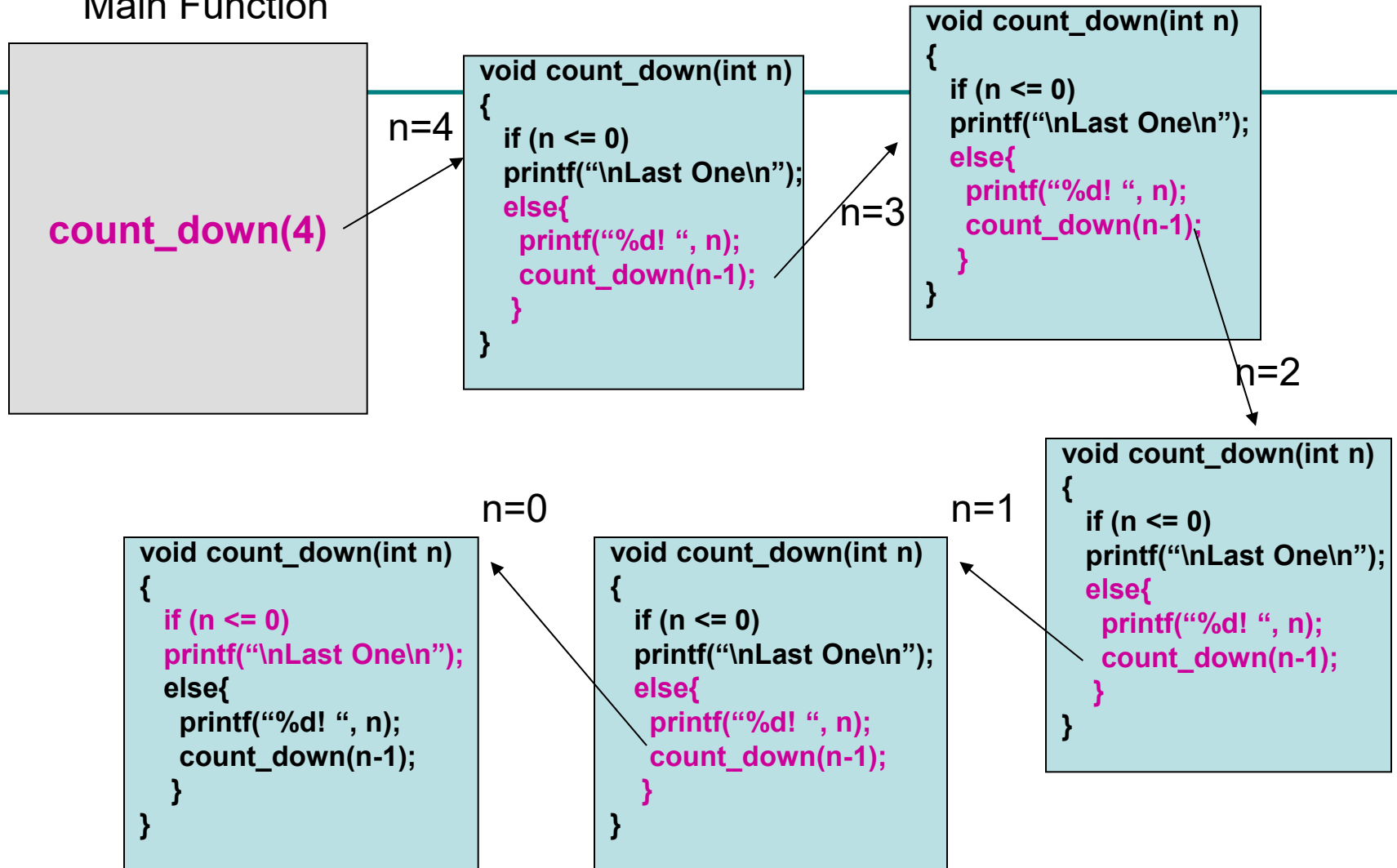
Output: 4 3 2

Main Function



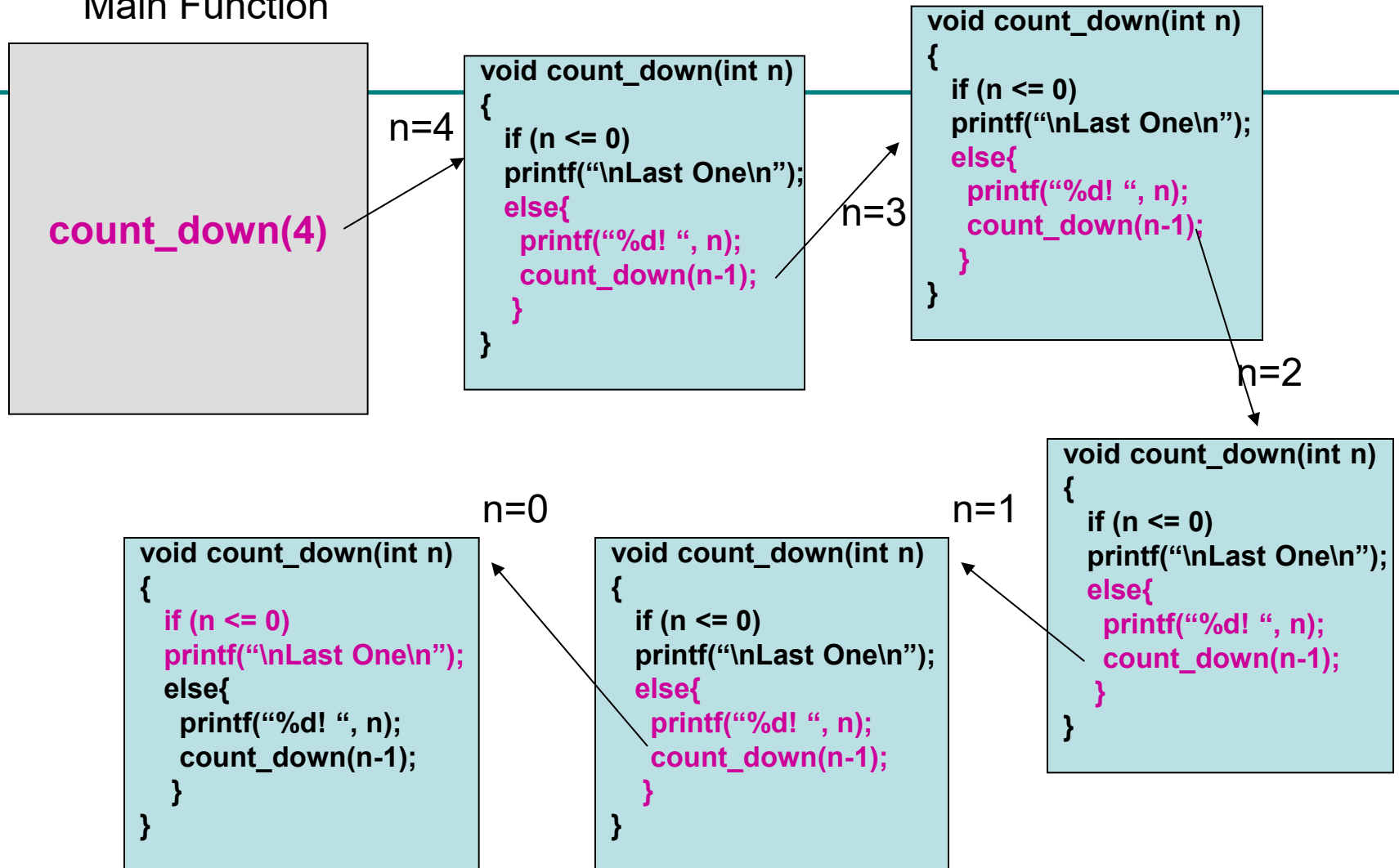
Output: 4 3 2 1

Main Function



Output: 4 3 2 1

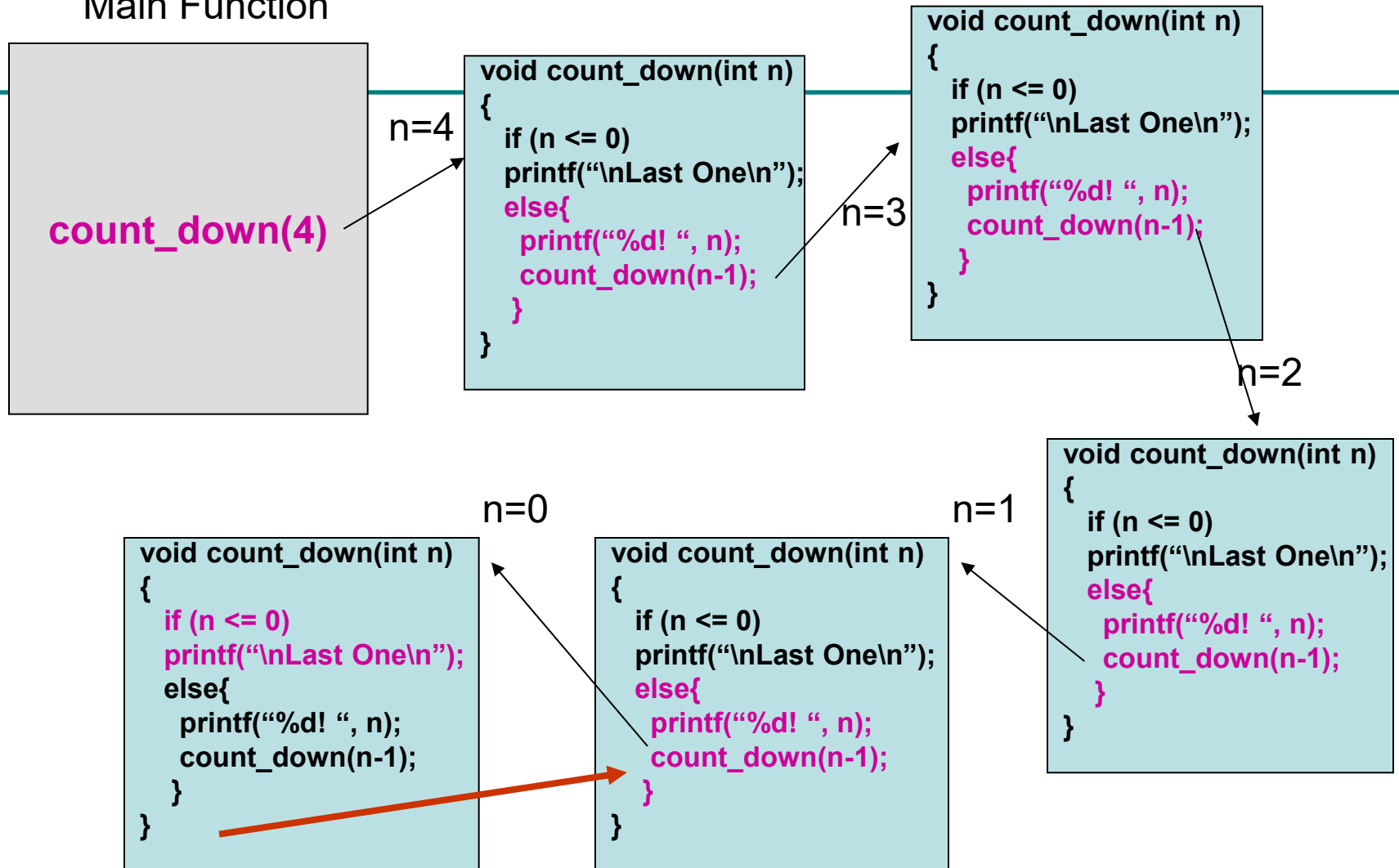
Main Function



Output: 4 3 2 1

Last One

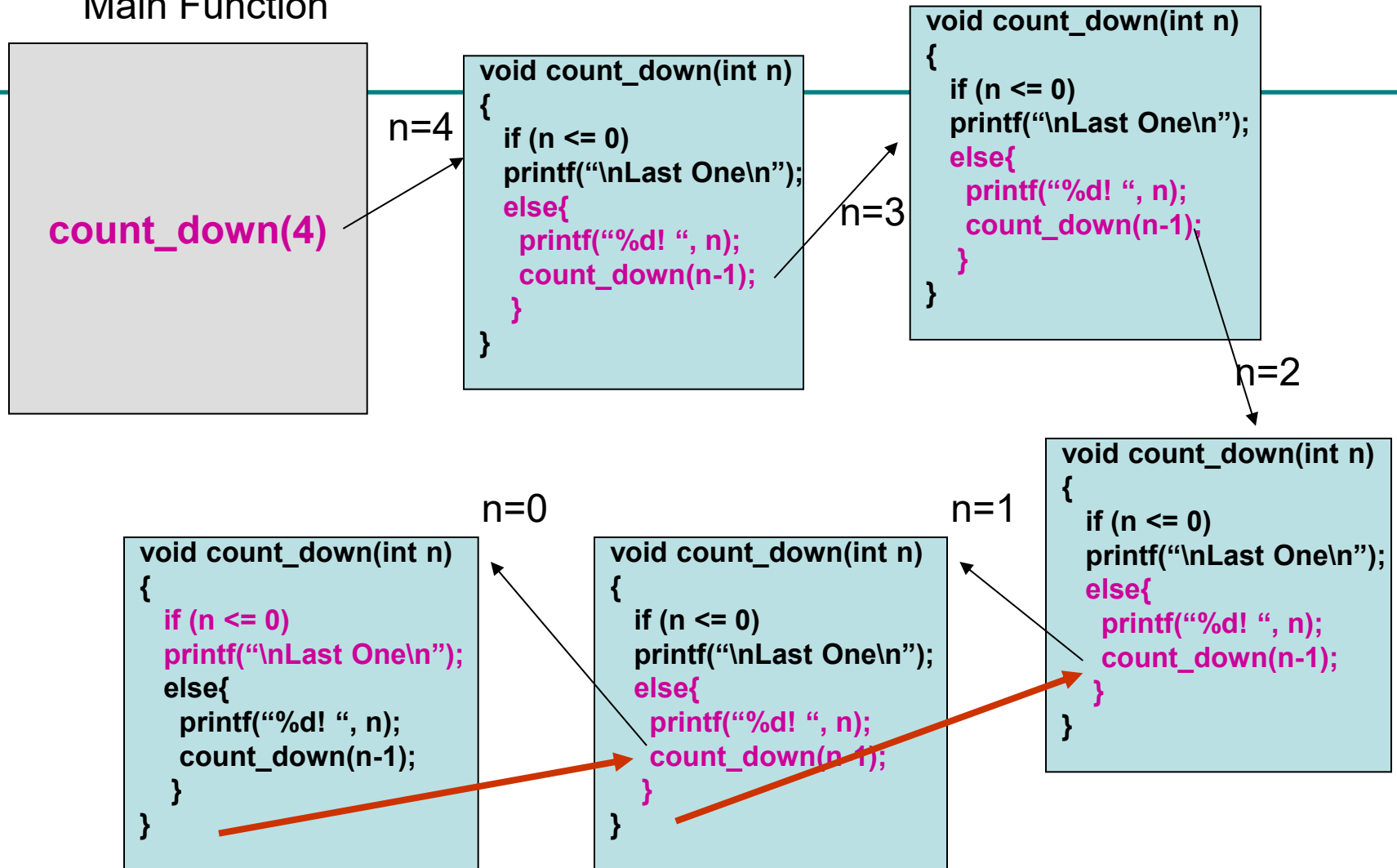
Main Function



Output: 4 3 2 1

Last One

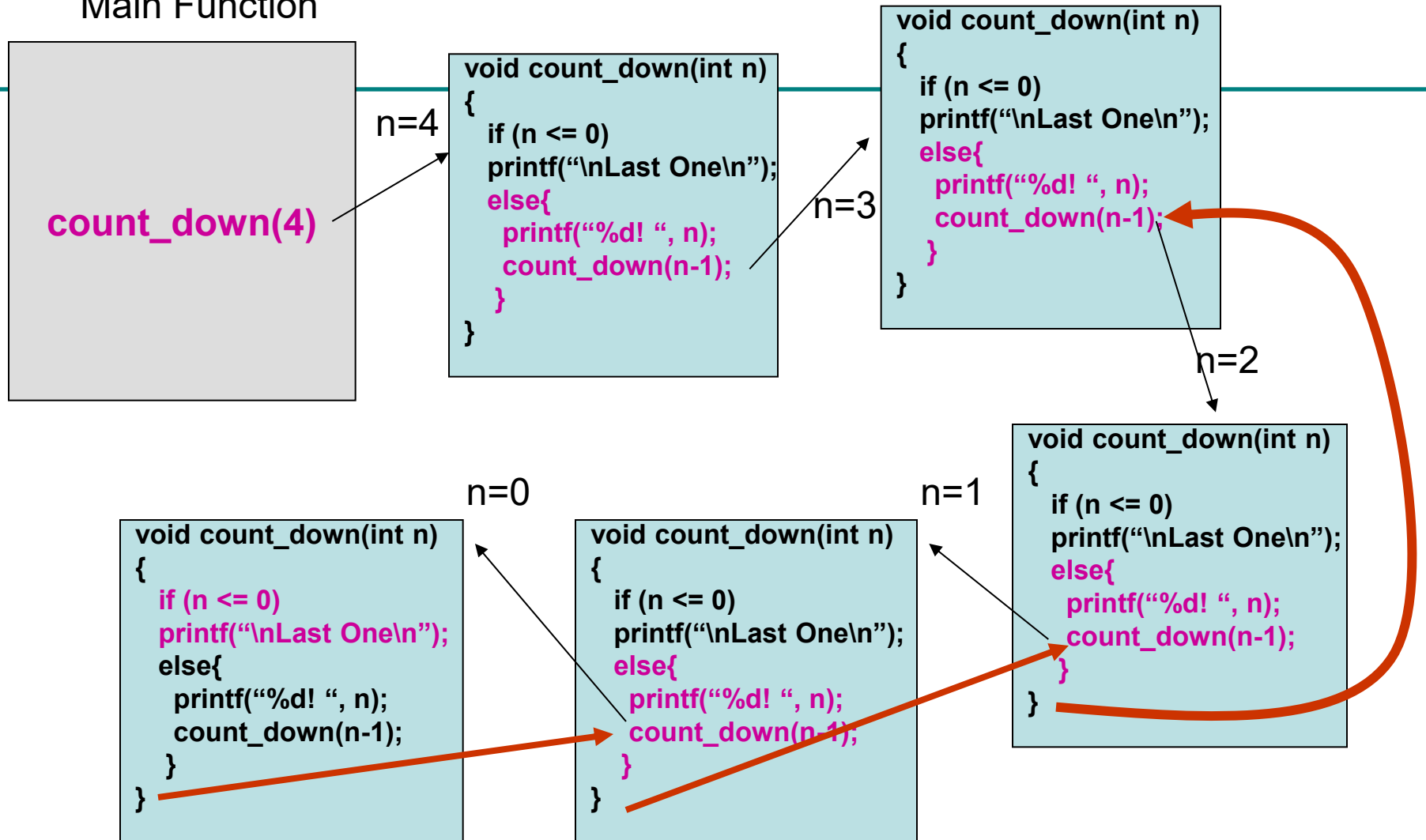
Main Function



Output: 4 3 2 1

Last One

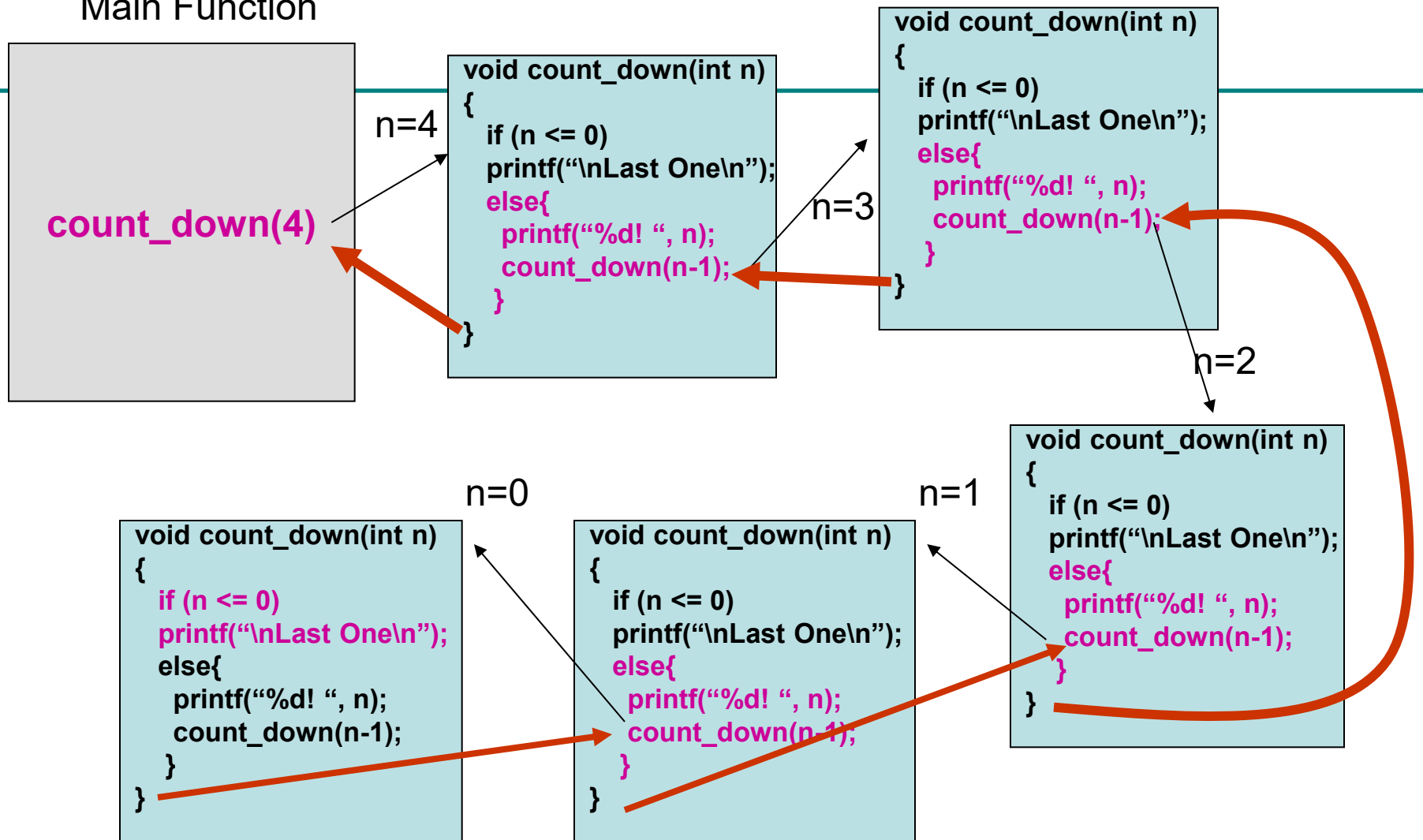
Main Function



Output: 4 3 2 1

Last One

Main Function



Output: 4 3 2 1

Last One

Recursion

To trace recursion, recall that function calls operate as a stack – the new function is put on top of the caller

We have to pay a price for recursion:

- ♦ Calling a function **consumes more time and memory** than adjusting a loop counter.
- ♦ High performance applications (graphic action games, simulations of nuclear explosions) hardly ever use recursion.

In less demanding applications recursion is an attractive alternative for iteration (for the right problems!)

Recursion

We must always make sure that the recursion *bottoms out*:

- A recursive function must contain **at least one non-recursive branch**.
- The recursive calls must eventually lead to a non-recursive branch.

How to Design

- Base case
 - ♦ The statement that "solves" the problem.
 - ♦ Every recursive algorithm must have a base case.
- General case
 - ♦ The rest of the algorithm
 - ♦ Contains the logic needed to reduce the size of the problem.

Every recursive call either solves a part of the problem or it reduce the size of the problem.

How to Design

- a. Determine the base case.
- b. Determine the general case.
- c. Combine the base case and the general cases into an algorithm.
 - ◆ Each call must reduce the size of the problem and move it toward the base case.
 - ◆ The base case, when reached, must terminate without a call to the recursive algorithms; that is, it must execute a return.

How to Design

Limitations of recursion

You should not use recursion if the answer to any of the following questions is **NO**:

- a. Is the algorithm or data structure naturally suited to recursion?
- b. Is the recursive solution shorter and more understandable?
- c. Does the recursive solution run within acceptable time and space limits?

Recursion

If we use iteration, we must be careful not to create an infinite loop by accident:

```
for(int incr=1;incr!=10;incr+=2)
```

```
...
```

```
int result = 1;
```

```
while(result >0) {
```

```
...
```

```
    result++;
```

```
}
```


Recursion

Similarly, if we use recursion we must be careful not to create an infinite chain of function calls:

```
int factorial(int numb) {  
    return numb * fac(numb-1);  
}
```

Or:

```
int factorial(int numb) {  
    if (numb==0) ←  
        return 1;  
    else  
        return numb * fac(numb-1);  
}
```



Example 2: Fibonacci numbers

- Fibonacci numbers:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

where each number is the sum of the preceding two.

Fibonacci numbers and nature:

<http://www.mcs.surrey.ac.uk/Personal/R.Knott/Fibonacci/fibnat.html>

- Recursive definition:

- ♦ `Fib(0) = 0;`

- ♦ `Fib(1) = 1;`

- ♦ `Fib(number) = Fib(number-1) + Fib(number-2);`

Example 2: Fibonacci numbers

```
//Calculate Fibonacci numbers using recursive function.  
//A very inefficient way, but illustrates recursion well
```

```
#include <stdlib.h>
```

```
int fib(int number)  
{  
    if (number == 0) return 0;  
    if (number == 1) return 1;  
    return (fib(number-1) + fib(number-2));  
}
```

```
int main(void)  
{  
    // driver function  
    int *pNumber;  
  
    pNumber = (int*)malloc(sizeof(int));  
    if(pNumber == NULL)  
        printf( "Out of Space!!!" );  
  
    printf( "Please enter an integer: " );  
    scanf( "%d", pNumber );  
    printf( "\nThe Fibonacci number for %d is: %d", *pNumber, fib(*pNumber) );  
  
    free(pNumber);  
  
    return 0;  
}
```

Result:

```
cur-src\Fib>fibonacci  
Please enter an integer: 1  
  
The Fibonacci number for 1 is: 1  
F:\MUST\Teaching\2008-09-1stTerm\cur-src\Fib>fibonacci  
Please enter an integer: 2  
  
The Fibonacci number for 2 is: 1  
F:\MUST\Teaching\2008-09-1stTerm\cur-src\Fib>fibonacci  
Please enter an integer: 3  
  
The Fibonacci number for 3 is: 2  
F:\MUST\Teaching\2008-09-1stTerm\cur-src\Fib>fibonacci  
Please enter an integer: 4  
  
The Fibonacci number for 4 is: 3  
F:\MUST\Teaching\2008-09-1stTerm\cur-src\Fib>fibonacci  
Please enter an integer: 5  
  
The Fibonacci number for 5 is: 5
```

Fibonacci numbers:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Trace a Fibonacci Number

- Assume the input number is 4, that is, num=4:

fib(4) :

4 == 0 ? No; 4 == 1? No.

fib(4) = fib(3) + fib(2)

fib(3) :

3 == 0 ? No; 3 == 1? No.

fib(3) = fib(2) + fib(1)

fib(2) :

2 == 0? No; 2==1? No.

fib(2) = fib(1)+fib(0)

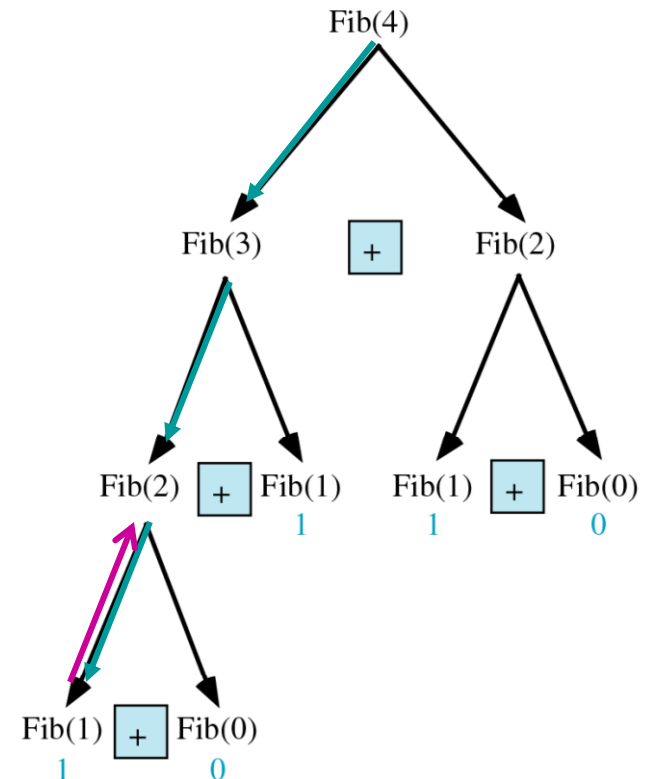
fib(1) :

1== 0 ? No; 1 == 1? Yes.

fib(1) = 1;

return fib(1) ;

```
int fib(int num)
{
    if (num == 0) return 0;
    if (num == 1) return 1;
    return
        (fib(num-1)+fib(num-2)) ;
}
```



Trace a Fibonacci Number

```
fib(0):
```

```
    0 == 0 ? Yes.
```

```
    fib(0) = 0;
```

```
    return fib(0);
```

```
fib(2) = 1 + 0 = 1;
```

```
return fib(2);
```

```
fib(3) = 1 + fib(1)
```

```
fib(1):
```

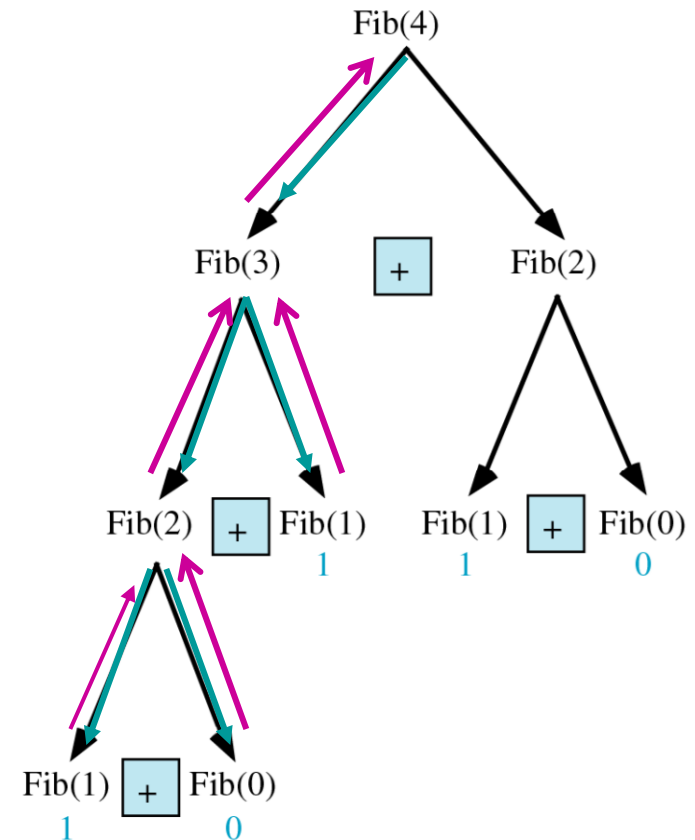
```
    1 == 0 ? No; 1 == 1? Yes
```

```
    fib(1) = 1;
```

```
    return fib(1);
```

```
fib(3) = 1 + 1 = 2;
```

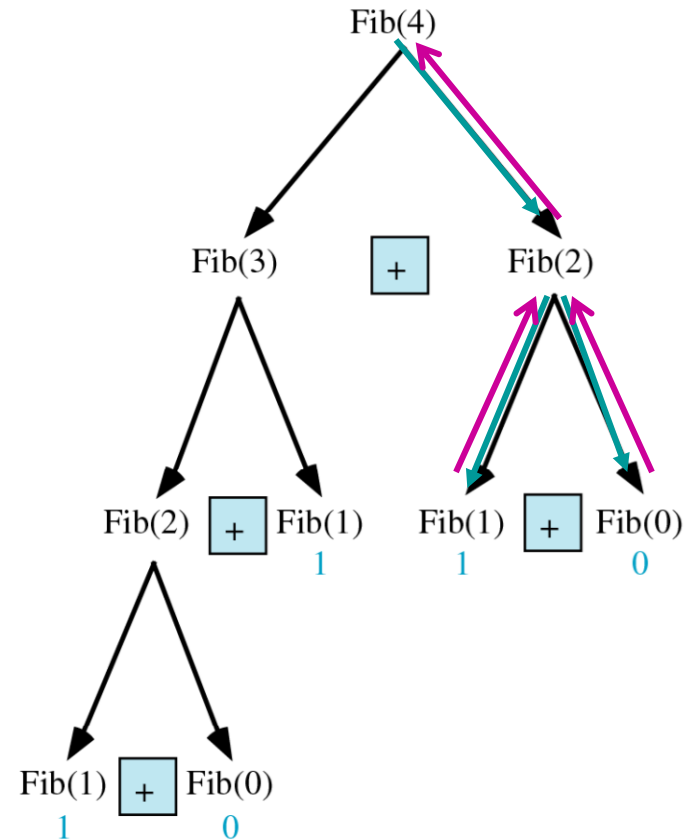
```
return fib(3)
```



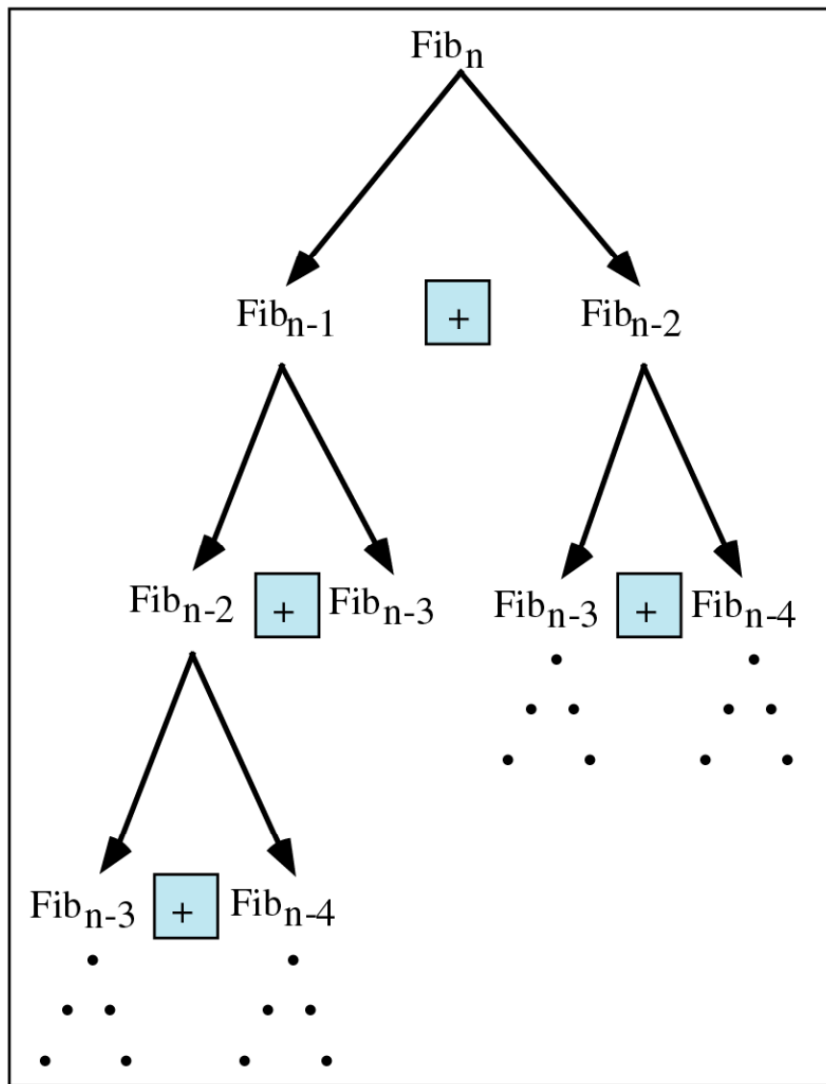
```
int fib(int num)
{
    if (num == 0) return 0;
    if (num == 1) return 1;
    return
        (fib(num-1)+fib(num-2));
}
```

Trace a Fibonacci Number

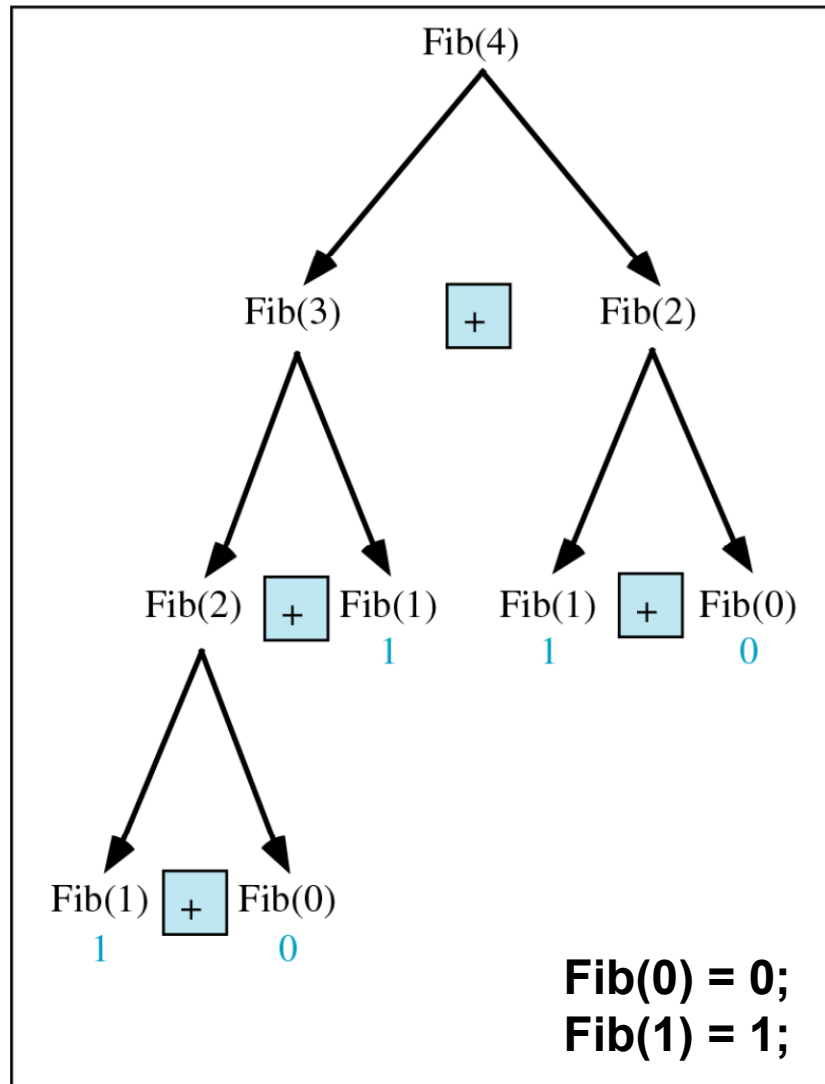
```
fib(2):  
  2 == 0 ? No; 2 == 1?   No.  
  fib(2) = fib(1) + fib(0)  
  fib(1):  
    1 == 0 ? No; 1 == 1?  Yes.  
    fib(1) = 1;  
    return fib(1);  
  fib(0):  
    0 == 0 ?   Yes.  
    fib(0) = 0;  
    return fib(0);  
  fib(2) = 1 + 0 = 1;  
  return fib(2);  
fib(4) = fib(3) + fib(2)  
       = 2 + 1 = 3;  
return fib(4);
```



```
int fib(int num)  
{  
  if (num == 0) return 0;  
  if (num == 1) return 1;  
  return  
    (fib(num-1)+fib(num-2));  
}
```



(a) $\text{Fib}(n)$



(b) $\text{Fib}(4)$

Fibonacci Number w/o Recursion

```
//Calculate Fibonacci numbers iteratively  
//much more efficient than recursive solution
```

```
int fib(int n)  
{  
    int f[n+1];  
    f[0] = 0; f[1] = 1;  
    for (int i=2; i<= n; i++)  
        f[i] = f[i-1] + f[i-2];  
    return f[n];  
}
```


Conclusion: The Nature of Recursion

- Recursion is one way to decompose a problem into smaller sub-problems that have **similar** or **identical structures**.
 - ♦ There must be at least one (or more) simple case of the problem (called the *stopping case*) to terminate the recursion, namely, a simple non-recursive solution.
 - ♦ The other cases of the problem can be reduced (*using recursion*) to problems that are closer to stopping cases.
 - ♦ Eventually the problem can be reduced to stopping cases only, which are relatively easy to solve.
- In general:
 - if (stopping case)*
 - solve it and return the stopping value*
 - else*
 - reduce the problem using recursion*

Assignment 06

- Write a recursive program to calculate the sum of $1, 2, \dots, n$.

Input “n” by keyboard and calculate the sum. (Don’t use too large n for test.)

- Submit it to Moodle.